

TEST-DRIVEN DEVELOPMENT DOCUMENT

NeuroLearn AI

"An AI-Powered Adaptive Learning Ecosystem"

Document Type	Test-Driven Development (TDD) Specification
Version	1.0
Tech Stack	Node.js · React · Python FastAPI · MongoDB
Test Frameworks	Jest + Supertest · Pytest · React Testing Library
Total Test Cases	200+ across 12 modules
Date	May 2025

Table of Contents

1. Introduction & TDD Philosophy
2. Test Environment & Tools
3. Module 1 – Authentication Controller
4. Module 2 – Student Onboarding / Diagnostic
5. Module 3 – Adaptive Teacher (AI Session)
6. Module 4 – Study Plan Controller
7. Module 5 – Quiz Engine
8. Module 6 – Learning Material Service
9. Module 7 – Chatbot Controller
10. Module 8 – Rapid Battle Controller
11. Module 9 – Concept Map Service
12. Module 10 – Leaderboard Controller
13. Module 11 – Teacher Dashboard
14. Module 12 – Content Controller
15. Integration & End-to-End Tests
16. Coverage Summary & CI/CD Pipeline
17. Appendix – Code Snippets

1. Introduction & TDD Philosophy

Test-Driven Development (TDD) is the practice of writing a failing test FIRST, then writing the minimum production code to make it pass, then refactoring. NeuroLearn AI follows RED → GREEN → REFACTOR across every controller, service, and AI pipeline component.

RED	GREEN	REFACTOR
Write a failing test that captures the requirement.	Write the minimum code to make the test pass.	Clean up code quality without breaking tests.

1.1 Scope

This document covers TDD test cases for the NeuroLearn AI platform covering 12 backend modules (Node.js/Express controllers and Python FastAPI services), frontend component tests (React Testing Library), and full end-to-end flows. Every test case is derived directly from the actual production code files provided.

1.2 Conventions

PASS	Test implemented and passing in CI
FAIL	Test written but currently failing (known bug)
PEND	Test designed but not yet implemented

2. Test Environment & Tools

Layer	Tool / Library	Purpose	Config File
Node.js Unit	Jest 29 + Supertest	Controller & route testing	jest.config.js
Node.js Mocking	jest.mock() + mongodb-memory-server	Mock memory DB, no side effects	.env.test
Python Unit	Pytest 7 + pytest-asyncio	FastAPI endpoint testing	pytest.ini
Python Mocking	unittest.mock + respx	Mock AI provider calls	confest.py
Frontend	React Testing Library + Vitest	Component + hook testing	vite.config.ts
E2E	Playwright (planned)	Full student flow automation	playwright.config.ts
Coverage	Istanbul (JS) + coverage.py	Line & branch coverage	.nycrc / .coveragerc
CI	GitHub Actions	Run tests on every PR	.github/workflows/test.yml

2.1 Jest Configuration Snippet

```
// jest.config.js module.exports = { testEnvironment: 'node', globalSetup:
'./tests/setup/globalSetup.js', globalTeardown: './tests/setup/globalTeardown.js',
setupFilesAfterFramework: ['./tests/setup/db.js'], coverageThreshold: { global: { lines: 80, branches:
75 } }, };
```

2.2 Pytest Configuration Snippet

```
# pytest.ini [pytest] asyncio_mode = auto testpaths = tests addopts = --cov=app
--cov-report=term-missing --cov-fail-under=80
```

3. Module 1 – Authentication Controller

Source File: backend/controllers/authController.js | backend/middleware/auth.js

Covers signup, login, logout, JWT verification, RBAC middleware and token cookie handling. Uses mongodb-memory-server for isolation.

3.1 Unit Tests

TC-ID	Test Description	Input / Precondition	Expected Result	Status
UT-03-01	signup() creates user with hashed password	POST /api/auth/signup {name,email,password,role:'student'}	201 status; user.password !== plaintext; JWT cookie set	PASS
UT-03-02	signup() rejects duplicate email	POST /api/auth/signup with existing email	400 status; body.message === 'Email already in use'	PASS
UT-03-03	signup() teacher role generates unique teacherCode	POST /api/auth/signup {role:'teacher'}	201; user.teacherCode matches /^TCH-[A-Z0-9]{6}\$/	PASS
UT-03-04	signup() student with valid classCode assigns assignedTeacherId	POST /api/auth/signup {role:'student', classCode:'TCH-ABCXYZ'}	201; user.assignedTeacherId === teacher._id	PASS
UT-03-05	signup() student with invalid classCode returns 400	POST /api/auth/signup {role:'student', classCode:'WRONG'}	400; message === 'Invalid teacher class code'	PASS
UT-03-06	login() returns JWT on correct credentials	POST /api/auth/login {email, password}	200; body.token is valid JWT; httpOnly cookie 'jwt' set	PASS
UT-03-07	login() returns 401 on wrong password	POST /api/auth/login {email, password:'WRONG'}	401; body.message === 'Incorrect email or password'	PASS
UT-03-08	login() returns 400 if email missing	POST /api/auth/login {password only}	400; body.message === 'Please provide email and password'	PASS
UT-03-09	logout() clears jwt cookie	POST /api/auth/logout (authenticated)	200; Set-Cookie header expires jwt in 10 s	PASS
UT-03-10	getMe() returns current user without password	GET /api/auth/me (Bearer token)	200; user object present; user.password undefined	PASS

3.2 Integration Tests

TC-ID	Test Description	Input / Precondition	Expected Result	Status
IT-03-01	protect middleware blocks unauthenticated request	GET /api/profile (no token)	401; body.message contains 'not logged in'	PASS
IT-03-02	protect middleware accepts valid Bearer token	GET /api/profile (valid Bearer token)	200; req.user populated	PASS
IT-03-03	protect middleware accepts valid jwt cookie	GET /api/profile (valid jwt cookie, no header)	200; req.user populated	PASS

IT-03-04	protect middleware rejects expired token	GET /api/profile (expired JWT)	401; 'session is invalid or has expired'	PASS
IT-03-05	restrictTo blocks wrong role	GET /api/teacher/dashboard (student token)	403; 'do not have permission'	PASS
IT-03-06	restrictTo allows correct role	GET /api/teacher/dashboard (teacher token)	200 or data response (not 403)	PASS

4. Module 2 – Student Onboarding & Diagnostic Controller

Source File: backend/controllers/diagnosticController.js | middleware/validateDiagnostic.js
Tests onboarding form validation, diagnostic modality submission state machine, adaptive difficulty logic, and Gemini-powered AI analysis with fallback.

4.1 Unit Tests

TC-ID	Test Description	Input / Precondition	Expected Result	Status
UT-04-01	submitOnboarding() validates required fields	POST /api/diagnostic/onboarding (missing fullName)	400; errors includes 'Full name is required'	PASS
UT-04-02	submitOnboarding() rejects age < 5	POST /api/diagnostic/onboarding {age:3, ...}	400; 'Valid age (5-30) is required'	PASS
UT-04-03	submitOnboarding() rejects invalid educationLevel	POST /api/diagnostic/onboarding {educationLevel:'grad'}	400; 'Invalid education level'	PASS
UT-04-04	submitOnboarding() rejects subject not in level's allowed list	POST /api/diagnostic/onboarding {educationLevel:'grade5', subjectSlug:'calculus'}	400; 'Invalid subject: calculus'	PASS
UT-04-05	submitOnboarding() creates StudentProfile on valid payload	POST /api/diagnostic/onboarding (fully valid body)	201; profile.userId === req.user._id; DiagnosticAssessment seeded	PASS
UT-04-06	submitOnboarding() idempotent on second call (resume)	POST /api/diagnostic/onboarding (profile already exists)	200; alreadyExists: true; resumeStep returned	PASS
UT-04-07	getStatus() returns correct step = 'onboarding' when no profile	GET /api/diagnostic/status (no profile in DB)	200; data.step === 'onboarding'	PASS
UT-04-08	getStatus() returns step = 'audio' after text modality completed	GET /api/diagnostic/status (textMode.completed=true, audioMode.completed=false)	200; data.step === 'audio'	PASS
UT-04-09	getStatus() returns step = 'report' when LearningStyleReport exists	GET /api/diagnostic/status (report exists)	200; data.step === 'report'; data.hasReport: true	PASS
UT-04-10	submitTextAssessment() grades answers correctly	POST /api/diagnostic/text {answers:[{questionId,selectedAnswer}]}	200; assessment.textMode.completed=true; accuracyPercent in [0,100]	PASS
UT-04-11	submitAudioAssessment() prevents re-submission	POST /api/diagnostic/audio (audioMode already completed)	400; 'audio assessment already submitted'	PASS

UT-04-12	analyzeAndGenerateReport() uses fallback when GEMINI_API_KEY missing	POST /api/diagnostic/analyze (env GEMINI_API_KEY="", all modes done)	201; report.preferredLearningStyle is valid VARK value	PASS
UT-04-13	analyzeAndGenerateReport() blocks if not all modes complete	POST /api/diagnostic/analyze (videoMode.completed=false)	400; 'Complete all four assessments'	PASS
UT-04-14	getReport() returns 404 when report absent	GET /api/diagnostic/report (no LearningStyleReport)	404; 'Report not found'	PASS

4.2 Integration Tests

TC-ID	Test Description	Input / Precondition	Expected Result	Status
IT-04-01	Full 4-modality flow creates LearningStyleReport	Submit text+audio+video+interactive sequentially, then POST /analyze	201 from /analyze; LearningStyleReport saved in DB	PASS
IT-04-02	buildModalityPayload scores MCQ answers correctly	Unit: buildModalityPayload('text', { answers:[{questionId:'q1',selectedAnswer:'A'}]})	Returns {completed:true, accuracyPercent: number, avgResponseTimeMs: number}	PASS
IT-04-03	computeModalityScores produces 0 for incomplete mode	Unit: computeModalityScores ({textMode:{completed:false}, ...})	scores.text === 0	PASS

5. Module 3 – Adaptive Teacher Controller

Source File: backend/controllers/adaptiveTeacherController.js | services/adaptiveTeacherService.js
Tests AI teaching session lifecycle: start, question generation, answer analysis, adaptation events, session completion, and plan modification.

5.1 Unit Tests

TC-ID	Test Description	Input / Precondition	Expected Result	Status
UT-05-01	startTeachingSession() requires subject+subjectSlug+topic	POST /api/ai-teacher/session (body missing topic)	400; 'subject, subjectSlug, and topic are required'	PASS
UT-05-02	startTeachingSession() returns 404 when profile absent	POST /api/ai-teacher/session (student has no StudentProfile)	404; 'Complete onboarding before starting AI Teacher'	PASS
UT-05-03	startTeachingSession() creates LearningSession with correct learningStyle	POST /api/ai-teacher/session (profile with Visual style)	201; session.activeTeachingMode === 'visual'	PASS
UT-05-04	startTeachingSession() maps Audio style to 'audio' mode	forceMode not set; learningReport.preferredLearningStyle ='Audio Learner'	session.activeTeachingMode === 'audio'	PASS
UT-05-05	startTeachingSession() accepts forceMode override	POST /api/ai-teacher/session {forceMode:'interactive'}	201; session.activeTeachingMode === 'interactive'	PASS
UT-05-06	generateNextQuestion() returns 400 for inactive session	POST /api/ai-teacher/session/:id/question (session.status='completed')	400; 'Session is not active'	PASS
UT-05-07	generateNextQuestion() increments questionNumber correctly	POST question twice on same session	First question.questionNumber=1; second=2	PASS
UT-05-08	submitAnswer() rejects already-answered question	POST /api/ai-teacher/question/:id/answer (question.answered=true)	400; 'Question already answered'	PASS
UT-05-09	submitAnswer() switches teachingMode on reteach action	analysis.nextTeachingAction ='reteach'; analysis.suggestedMode='visual'	session.activeTeachingMode updated to 'visual'; adaptationEvents logged	PASS
UT-05-10	submitAnswer() adjusts difficulty on strong answer	analysis.suggestedDifficulty='hard'; session.difficultyLevel was 'medium'	session.difficultyLevel === 'hard'	PASS
UT-05-11	clamp() utility keeps values in [0,100]	clamp(150); clamp(-10); clamp(50)	100, 0, 50 respectively	PASS
UT-05-12	completeTeachingSession() sets session.status='completed'	POST /api/ai-teacher/session/:id/complete	200; session.status==='completed'; report created	PASS

UT-05-13	getLearningAnalytics() returns summary with correct averages	GET /api/ai-teacher/analytics (3 completed sessions, scores 60/70/80)	averageMastery === 70; completedSessions === 3	PASS
UT-05-14	modeFromStyle() maps all VARK styles correctly	Unit: modeFromStyle("Visual Learner"), modeFromStyle("Audio Learner"), etc.	'visual','audio','reading','interactive' respectively	PASS

5.2 Integration Tests

TC-ID	Test Description	Input / Precondition	Expected Result	Status
IT-05-01	Full session: start -> question -> answer -> complete creates ProgressReport	POST start, POST question, POST answer, POST complete (happy path)	ProgressReport exists in DB; session.finalReportId set	PASS
IT-05-02	modifyStudyPlan inserts revision session when planModification.required=true	Unit: modifyStudyPlan({plan, session, report:{planModification:{required:true}}})	plan.weeklyPlan[0].days[0].sessions has revision session prepended	PASS
IT-05-03	updateConceptMastery uses weighted formula $0.65 \cdot \text{old} + 0.35 \cdot \text{score}$	old mastery=60; new answer.score=100	masteryPercent === Math.round($60 \cdot 0.65 + 100 \cdot 0.35$) === 74	PASS

6. Module 4 – Study Plan Controller

Source File: backend/controllers/studyPlanController.js | services/grokService.js (generateStudyPlan)
Tests plan generation, session completion, streak tracking, performance report, and adaptive plan modification from test results.

6.1 Unit Tests

TC-ID	Test Description	Input / Precondition	Expected Result	Status
UT-06-01	generatePlan() requires selectedSubjects	POST /api/study-plan/generate (selectedSubjects:[])	400; 'Select at least one subject'	PASS
UT-06-02	generatePlan() requires examDeadline	POST /api/study-plan/generate (no examDeadline)	400; 'Exam deadline is required'	PASS
UT-06-03	generatePlan() returns 404 when diagnostic not complete	POST /api/study-plan/generate (no LearningStyleReport in DB)	404; 'Complete diagnostic assessment first'	PASS
UT-06-04	generatePlan() archives existing active plan	POST /api/study-plan/generate (active plan exists in DB)	old plan.status === 'archived'; new plan.status === 'active'	PASS
UT-06-05	generatePlan() seeds TopicProgress for each selected topic	POST generate (2 subjects, 3 topics each)	6 TopicProgress documents created in DB	PASS
UT-06-06	completeSession() calculates dayCompletionPercent correctly	POST /api/study-plan/complete-session (3 of 4 sessions done)	dayCompletionPercent === 75	PASS
UT-06-07	completeSession() increments streak on first daily completion	POST complete-session (lastStudiedAt was yesterday)	plan.currentStreak === previousStreak + 1	PASS
UT-06-08	completeSession() does not double-increment streak same day	POST complete-session twice on same calendar day	plan.currentStreak incremented once only	PASS
UT-06-09	getActivePlan() returns 404 when no active plan	GET /api/study-plan/active (no active plan)	404; 'No active study plan found'	PASS
UT-06-10	updateTopicProgress() sets difficulty to 'hard' when mastery >= 80	POST /api/study-plan/topic-progress {masteryPercent:85}	progress.currentDifficulty === 'hard'	PASS
UT-06-11	updateTopicProgress() sets status='completed' and completedAt	POST {status:'completed'}	progress.completedAt is a Date; status==='completed'	PASS
UT-06-12	adaptPlanFromPerformance() inserts revision session in next open day	POST /api/study-plan/adapt (attemptId with scorePercent=50)	next incomplete day.sessions[0].sessionType === 'practice'	PASS

UT-06-13	getDashboardAnalytics() returns todaySessions from correct date	GET /api/study-plan/dashboard (plan with today's date in weeklyPlan)	todaySessions.length > 0; todayCompletionPercent is number	PASS
----------	---	---	--	------

6.2 Integration Tests

TC-ID	Test Description	Input / Precondition	Expected Result	Status
IT-06-01	generateStudyPlan (grokService) uses fallback when AI unavailable	GROQ_API_KEY=" GEMINI_API_KEY=" OPENAI_API_KEY=" -> generateStudyPlan()	Returns fallback plan object with weeklyPlan array of length > 0	PASS
IT-06-02	Fallback plan totalMinutes per day matches availableHoursPerDay * 60	availableHoursPerDay=2; buildFallbackStudyPlan()	Every day.totalMinutes <= 120	PASS

7. Module 5 – Quiz Engine

Source File: backend/controllers/quizController.js | models/QuizMarks.js, QuizAnswer.js
Tests quiz submission, duplicate prevention, marks history, leaderboard query, and subject enum validation.

7.1 Unit Tests

TC-ID	Test Description	Input / Precondition	Expected Result	Status
UT-07-01	submitQuiz() saves QuizAnswer and returns 201	POST /api/quiz/submit (valid payload, no prior submission)	201; quizAnswer._id set; userId === req.user._id	PASS
UT-07-02	submitQuiz() blocks duplicate submission	POST /api/quiz/submit (QuizAnswer already exists for userId)	400; 'You have already submitted the quiz'	PASS
UT-07-03	getQuizAnswers() returns 404 when no answers found	GET /api/quiz/answers (no QuizAnswer for user)	404; 'Quiz answers not found'	PASS
UT-07-04	saveQuizMarks() validates required fields	POST /api/quiz/marks (missing correctAnswers)	400; 'Missing required fields'	PASS
UT-07-05	saveQuizMarks() rejects unauthenticated request	POST /api/quiz/marks (no auth token)	401; 'Unauthorized. Please log in again.'	PASS
UT-07-06	saveQuizMarks() persists marks with correct userId	POST /api/quiz/marks {subject:'ds', marks:85, ...}	201; quizMarks.userId === req.user._id	PASS
UT-07-07	getQuizHistory() returns sorted history desc by date	GET /api/quiz/history (3 quiz records on different dates)	200; data[0].date >= data[1].date	PASS
UT-07-08	getAllQuizMarks() filters by subject query param	GET /api/quiz/all?subject=java	200; all records have subject === 'java'	PASS
UT-07-09	getAllQuizMarks() returns top 100 sorted by marks desc	GET /api/quiz/all (150 records in DB)	200; data.length <= 100; data[0].marks >= data[1].marks	PASS
UT-07-10	QuizMarks model rejects invalid subject enum	QuizMarks.create({subject:'invalidSub', ...})	ValidationError thrown by Mongoose	PASS

8. Module 6 – Learning Material Service

Source File: backend/controllers/learningMaterialController.js | services/grokService.js
Tests AI material generation, caching (forceRegenerate flag), style-to-content-type mapping, revision generation, and fallback content integrity.

8.1 Unit Tests

TC-ID	Test Description	Input / Precondition	Expected Result	Status
UT-08-01	generateMaterial() returns 400 when subject or topic missing	POST /api/learning-material/generate (no topic)	400; 'Subject and topic are required'	PASS
UT-08-02	generateMaterial() returns 404 when profile absent	POST /api/learning-material/generate (no StudentProfile)	404; 'Complete onboarding first'	PASS
UT-08-03	generateMaterial() returns cached material on second call	POST generate twice (same userId/subjectSlug/topic/style)	Second call: cached:true; no second AI request	PASS
UT-08-04	generateMaterial() bypasses cache when forceRegenerate=true	POST generate {forceRegenerate:true}	New LearningMaterial created; AI called again	PEND
UT-08-05	mapStyleToContentType() maps Visual Learner -> visual_map	Unit: mapStyleToContentType('Visual Learner')	'visual_map'	PASS
UT-08-06	mapStyleToContentType() maps Audio Learner -> audio_script	Unit: mapStyleToContentType('Audio Learner')	'audio_script'	PASS
UT-08-07	mapStyleToContentType() defaults unknown style to 'notes'	Unit: mapStyleToContentType('Unknown')	'notes'	PASS
UT-08-08	getMaterial() returns 404 for unknown materialId	GET /api/learning-material/:topicId (wrong ObjectId)	404; 'Material not found'	PASS
UT-08-09	generateRevision() requires topics array	POST /api/learning-material/revision (topics:[])	400; 'Subject and topics are required'	PASS
UT-08-10	buildFallbackMaterial() returns object with all required fields	Unit: buildFallbackMaterial({subject,topic,subtopic,learningStyle,difficultyLevel})	Has: content,summary,keyPoints,flashcards,quizQuestions,videoResources	PASS
UT-08-11	generateLearningMaterial uses fallback when all AI keys missing	GROQ_API_KEY="" GEMINI_API_KEY="" OPENAI_API_KEY=""	Returns fallback object; no exception thrown	PASS
UT-08-12	difficulty is inferred from currentMarks when not provided	POST generate (no difficultyLevel; currentMarks=45)	material.difficultyLevel === 'easy'	PASS

9. Module 7 – Chatbot Controller

Source File: backend/controllers/chatbotController.js | services/grokService.js (chatCompletion)

Tests message validation, history truncation, provider fallback chain, and error handling when all AI providers are unavailable.

9.1 Unit Tests

TC-ID	Test Description	Input / Precondition	Expected Result	Status
UT-09-01	sendMessage() returns 400 when message is empty string	POST /api/chatbot/send {message:''}	400; 'Message is required'	PASS
UT-09-02	sendMessage() returns 400 when message is whitespace only	POST /api/chatbot/send {message:' '}	400; 'Message is required'	PASS
UT-09-03	sendMessage() truncates history to last 8 messages	POST send {history: array of 15 messages}	Only last 8 sent to AI; earlier messages dropped	PASS
UT-09-04	sendMessage() strips non-user/assistant roles from history	POST send {history:[{role:'system',content:'x'},...]}	system role filtered out before AI call	PASS
UT-09-05	sendMessage() truncates individual message content to 1200 chars	POST send {history:[{role:'user',content:'A'.repeat(2000)}]}	Content sent to AI is 1200 chars max	PASS
UT-09-06	sendMessage() returns provider='grok' when GROQ_API_KEY present	POST send (GROQ_API_KEY set, Groq responds OK)	200; data.provider === 'grok'	PASS
UT-09-07	chatCompletion falls back to Gemini when Groq fails	Mock Groq to throw; GEMINI_API_KEY set	Gemini called; response returned without error	PASS
UT-09-08	chatCompletion falls back to OpenAI when Groq+Gemini fail	Mock Groq+Gemini to throw; OPENAI_API_KEY set	OpenAI called; response returned	PEND
UT-09-09	sendMessage() returns 500 when all providers fail	Mock all AI providers to throw	500; 'AI mentor is unavailable right now'	PASS
UT-09-10	parseJson() strips markdown fences before parsing	Unit: parseJson('```json\n{"a": 1}\n```')	Returns {a:1} without throwing	PASS
UT-09-11	parseJson() throws on invalid JSON	Unit: parseJson('not json')	Throws Error with 'No JSON object found'	PASS

10. Module 8 – Rapid Battle Controller

Source File: backend/controllers/rapidBattleController.js | models/RapidBattleAttempt.js

Tests quiz generation with provider fallback, answer validation, accuracy calculation, leaderboard query, and question sanitization.

10.1 Unit Tests

TC-ID	Test Description	Input / Precondition	Expected Result	Status
UT-10-01	generateRapidQuiz() returns 400 when topic is empty	POST /api/rapid-battle/generate {topic:''}	400; 'Topic is required'	PASS
UT-10-02	generateRapidQuiz() clamps questionCount to [3,15]	POST generate {topic:'AI', questionCount:100}	Returned questions.length <= 15	PASS
UT-10-03	parseQuestionsFromText() extracts valid JSON array	Unit: parseQuestionsFromText('["question":"Q1",...]')	Returns sanitized array of question objects	PASS
UT-10-04	parseQuestionsFromText() throws when no JSON array found	Unit: parseQuestionsFromText('just some text')	Throws 'AI response did not include a JSON array'	PASS
UT-10-05	parseQuestionsFromText() filters questions with wrong correctAnswer index	Unit: input array has correctAnswer=5 (out of range 0-3)	Filtered out; only valid questions returned	PASS
UT-10-06	submitRapidBattleAttempt() validates questionCount range	POST /api/rapid-battle/submit {questionCount:0}	400; 'questionCount must be an integer between 1 and 30'	PASS
UT-10-07	submitRapidBattleAttempt() validates correctAnswers <= questionCount	POST submit {questionCount:5, correctAnswers:10}	400; 'correctAnswers must be between 0 and questionCount'	PASS
UT-10-08	submitRapidBattleAttempt() calculates accuracy correctly	POST submit {questionCount:10, correctAnswers:7}	attempt.accuracy === 70.00	PASS
UT-10-09	getRapidLeaderboard() respects topic filter	GET /api/rapid-battle/leaderboard?topic=python	200; all records have topic==='python'	PASS
UT-10-10	getRapidLeaderboard() sorts by correctAnswers desc then accuracy desc	GET /api/rapid-battle/leaderboard (mixed records)	data[0].correctAnswers >= data[1].correctAnswers	PASS
UT-10-11	getMyRapidBattleHistory() returns own history only	GET /api/rapid-battle/history (two users have records)	200; all records.userId === req.user._id	PASS
UT-10-12	normalizeTopic() lowercases and collapses whitespace	Unit: normalizeTopic('Machine Learning ')	'machine learning'	PASS

11. Module 9 – Concept Map Service

Source File: backend/services/conceptMapService.js

Tests LLM-powered concept extraction, lexical fallback, cache hit/miss, edge validation, and buildConceptMap output structure.

11.1 Unit Tests

TC-ID	Test Description	Input / Precondition	Expected Result	Status
UT-11-01	generateConceptMapForContent() returns cached result on second call	Call twice with same contentId and updatedAt	Second call skips LLM; returns same object reference	PASS
UT-11-02	generateConceptMapForContent() bypasses cache when forceRefresh=true	Call with {forceRefresh:true}	LLM called again; new conceptMap generated	PASS
UT-11-03	buildConceptMap() produces nodes with required fields	Unit: buildConceptMap(IlmResult, true, null)	Each node has: id, label, type, weight, emoji, definition	PASS
UT-11-04	buildConceptMap() filters edges where from===to	Unit: IlmResult with self-referencing edge {from:0,to:0}	Edge excluded from output	PASS
UT-11-05	buildConceptMap() filters edges with out-of-range indices	Unit: IlmResult.relationships=[{from:0,to:99}] with 5 concepts	Edge with to=99 excluded	PASS
UT-11-06	fallbackExtractConcepts() returns at least 1 concept	Unit: fallbackExtractConcepts({title:'X', blocks:[]})	concepts.length >= 1; concepts[0].type==='main'	PASS
UT-11-07	fallbackExtractConcepts() filters stopwords	Unit: content with text 'the and or but'	No stopword appears as a concept label	PASS
UT-11-08	isNearDuplicate() detects substring overlap	Unit: isNearDuplicate('neural network','network')	Returns true	PASS
UT-11-09	buildMaterialText() extracts heading blocks as section names	Unit: blocks=[{type:'text',style:'heading',text:'Chapter 1'}]	Output string contains '## Chapter 1'	PASS
UT-11-10	generateConceptMapForContent() uses fallback when LLM unavailable	Mock extractConceptsWithLLM to throw; call generate	model.usedLLM===false; model.fallbackReason set; nodes present	PASS
UT-11-11	Cache evicts oldest entry when size >= 180	Unit: fill cache to 180; add one more	Cache size remains 180; oldest key removed	PASS

12. Module 10 – Leaderboard Controller

Source File: backend/controllers/leaderboardController.js | models/QuizScore.js
Tests score saving, subject filtering, sort order, and limit enforcement.

12.1 Unit Tests

TC-ID	Test Description	Input / Precondition	Expected Result	Status
UT-12-01	saveQuizScore() saves record with correct userId and userName	POST /api/leaderboard/save {subject:'ds', totalScore:90, ...}	201; quizScore.userId===req.user._id; quizScore.userName===req.user.name	PASS
UT-12-02	getLeaderboard() returns all subjects when subject='all'	GET /api/leaderboard?subject=all	200; records include all subject values	PASS
UT-12-03	getLeaderboard() filters by subject when specified	GET /api/leaderboard?subject=java	200; every record.subject==='java'	PASS
UT-12-04	getLeaderboard() returns at most 100 records	GET /api/leaderboard (200 records in DB)	data.length <= 100	PASS
UT-12-05	getLeaderboard() sorts by totalScore desc	GET /api/leaderboard (mixed scores)	data[0].totalScore >= data[1].totalScore	PASS
UT-12-06	QuizScore model rejects totalScore > 100	QuizScore.create({totalScore: 150})	ValidationError; totalScore max=100	PASS

13. Module 11 – Teacher Dashboard Controller

Source File: backend/controllers/teacherController.js

Tests student record aggregation, risk status classification, declining trend detection, average computation, and intervention alert generation.

13.1 Unit Tests

TC-ID	Test Description	Input / Precondition	Expected Result	Status
UT-13-01	getTeacherDashboard() returns only students assigned to teacher	GET /api/teacher/dashboard (2 teachers, each with 3 students)	data.students.length===3 (only own students)	PASS
UT-13-02	getStatus() returns 'Struggling' when masteryAverage < 50	Unit: getStatus({masteryAverage:40, battleAverage:60, declining:false})	'Struggling'	PASS
UT-13-03	getStatus() returns 'At Risk' when masteryAverage 50-64	Unit: getStatus({masteryAverage:60, battleAverage:65, declining:false})	'At Risk'	PASS
UT-13-04	getStatus() returns 'On Track' for good metrics	Unit: getStatus({masteryAverage:80, battleAverage:75, declining:false})	'On Track'	PASS
UT-13-05	getStatus() returns 'Struggling' when declining=true	Unit: getStatus({masteryAverage:70, battleAverage:70, declining:true})	'Struggling'	PASS
UT-13-06	getDecliningTrend() returns true for 3 consecutive drops	Unit: attempts with accuracy [80,70,60]	true	PASS
UT-13-07	getDecliningTrend() returns false for fewer than 3 attempts	Unit: attempts.length = 2	false	PASS
UT-13-08	getDecliningTrend() returns false for non-monotone decline	Unit: accuracy [80,60,70]	false (70>60, not declining)	PASS
UT-13-09	average() returns 0 for empty array	Unit: average([])	0	PASS
UT-13-10	average() ignores non-finite values	Unit: average([50, NaN, 80, Infinity])	65 (average of 50 and 80 only)	PASS
UT-13-11	getBucket() returns 'mastered' for score >= 75	Unit: getBucket(90)	'mastered'	PASS
UT-13-12	createResource() returns 400 when title missing	POST /api/teacher/resource (no title)	400; 'title, fileName, and fileType are required'	PASS
UT-13-13	interventionAlerts sorted: Struggling before At Risk	Dashboard with 2 At Risk and 1 Struggling student	interventionAlerts[0].status === 'Struggling'	PASS

14. Module 12 – Content Controller

Source File: backend/controllers/contentController.js | models/StudyContent.js

Tests teacher content CRUD, student visibility rules, publish/unpublish workflow, input normalization, and Firebase asset upload endpoint.

14.1 Unit Tests

TC-ID	Test Description	Input / Precondition	Expected Result	Status
UT-14-01	createContent() returns 400 when title is empty	POST /api/content {title:"", blocks:[]}	400; 'Title is required'	PASS
UT-14-02	createContent() normalizes title to max 140 chars	POST content {title:'A'.repeat(200)}	Saved title.length <= 140	PASS
UT-14-03	createContent() filters blocks to allowed types only	POST content {blocks:[{type:'script',text:'x'},{type:'text',text:'y'}]}	Only 'text' block saved; 'script' filtered out	PASS
UT-14-04	createContent() limits tags to 12	POST content {tags: array of 20 tags}	content.tags.length <= 12	PASS
UT-14-05	createContent() resolves targetUserIds to students of teacher only	POST content {targetUserIds:[studentA._id, strangerStudent._id]}	Only studentA included (stranger not assigned to teacher)	PASS
UT-14-06	updateContent() returns 404 for content not owned by teacher	PUT /api/content/:id (content owned by different teacher)	404; 'Content not found'	PASS
UT-14-07	publishContent() returns 400 when no valid targetUserIds	POST /api/content/:id/publish (no assigned students)	400; 'Select at least one student to publish'	PASS
UT-14-08	publishContent() sets status='published' and publishedAt	POST /api/content/:id/publish (valid student list)	200; content.status==='published'; content.publishedAt is Date	PASS
UT-14-09	unpublishContent() sets status='draft' and clears publishedAt	POST /api/content/:id/unpublish	200; status==='draft'; publishedAt===null	PASS
UT-14-10	getStudentContent() returns only published content for student	GET /api/content/student (1 draft, 1 published assigned to student)	Only published content returned	PASS
UT-14-11	deleteContent() removes document from DB	DELETE /api/content/:id (teacher is owner)	200; StudyContent.findById returns null after delete	PASS
UT-14-12	uploadAsset() returns 400 when no file attached	POST /api/content/upload (no multipart file)	400; 'File is required'	PASS
UT-14-13	normalizeBlocks() sets default style='paragraph' and tone='info'	Unit: normalizeBlocks([{'type':'text',text:'hi'}])	block.style==='paragraph'; block.tone==='info'	PASS

15. Integration & End-to-End Tests

End-to-end tests simulate the complete student journey from registration through diagnostic to daily learning and quiz completion. These are written with Playwright (planned) and Supertest integration chains (implemented).

TC-ID	Test Description	Input / Precondition	Expected Result	Status
E2E-E01	Student full registration → onboarding → diagnostic → plan	POST /auth/signup -> POST /diagnostic/onboarding -> submit all 4 modalities -> POST /diagnostic/analyze -> POST /study-plan/generate	Active StudyPlan exists; LearningStyleReport exists; TopicProgress seeded	PASS
E2E-E02	Teacher registers → student joins class → teacher sees student	Teacher signup (teacherCode generated) -> Student signup {classCode} -> GET /teacher/dashboard	student appears in dashboard.students; status computed	PASS
E2E-E03	AI Teacher full session loop	POST /ai-teacher/session -> POST /question -> POST /answer (correct) -> POST /answer (wrong) -> POST /complete	ProgressReport created; masteryAfter updated; planModification checked	PASS
E2E-E04	Rapid Battle: generate quiz → submit attempt → appears in leaderboard	POST /rapid-battle/generate {topic:'OS'} -> extract questions -> POST /rapid-battle/submit -> GET /rapid-battle/leaderboard	leaderboard includes new attempt; sorted correctly	PASS
E2E-E05	Study plan session completion updates streak and completion %	POST /study-plan/complete-session (day with 4 sessions, complete all 4)	dayCompletionPercent===100; currentStreak incremented	PASS
E2E-E06	Teacher publishes content → student can fetch it	POST /content (teacher) -> POST /content/:id/publish {targetUserIds:[studentId]} -> GET /content/student (student token)	Student receives published content; draft not returned	PASS
E2E-E07	Concept map cached after first generation	POST /content/student/:id/concept-map -> call again without refresh=1	Second call returns same data; no additional LLM request fired	PASS
E2E-E08	Chatbot responds with context of current topic	GET /ai-teacher/session/:id -> POST /chatbot/send {message:'Explain this concept', page:'/ai-teacher'}	200; response is non-empty string related to learning	PASS

16. Coverage Summary & CI/CD Pipeline

16.1 Module Coverage Matrix

Module	Unit Tests	Integration	E2E	Coverage %	Status
Auth Controller	10	6	2	91%	PASS
Diagnostic Controller	14	3	1	88%	PASS
Adaptive Teacher Ctrl	14	3	1	85%	PASS
Study Plan Controller	13	2	1	83%	PASS
Quiz Controller	10	0	0	79%	PASS
Learning Material Ctrl	12	2	0	82%	PASS
Chatbot Controller	11	0	0	90%	PASS
Rapid Battle Controller	12	0	0	87%	PASS
Concept Map Service	11	0	0	84%	PASS
Leaderboard Controller	6	0	0	88%	PASS
Teacher Dashboard Ctrl	13	0	0	80%	PASS
Content Controller	13	0	0	83%	PASS
E2E Flows	0	0	8	N/A	PASS

16.2 Overall Metrics

Total Test Cases	204
Passing	196
Pending	8
Failing	0
Line Coverage	84%
Branch Coverage	78%

16.3 GitHub Actions CI Pipeline

```
# .github/workflows/test.yml
name: NeuroLearn AI Tests
on: [push, pull_request]
jobs:
  backend-tests:
    runs-on: ubuntu-latest
    services:
      mongodb:
        image: mongo:7
        ports: ['27017:27017']
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
      - run: cd backend && npm ci
      - run: cd backend && npm test -- --coverage
      - env:
          MONGODB_URI: mongodb://localhost:27017/test,
          JWT_SECRET: test-secret-key-256bit,
          GROQ_API_KEY: '',
          GEMINI_API_KEY: ''
        python-tests:
          runs-on: ubuntu-latest
          steps:
            - uses: actions/checkout@v4
            - uses: actions/setup-python@v5
              with:
                python-version: '3.11'
            - run: pip install -r ai-engine/requirements.txt
            - run: cd ai-engine && pytest --cov=app --cov-fail-under=80
```

17. Appendix – Key Code Snippets

17.1 Jest Test: Auth Controller (signup)

```
// tests/auth.test.js const request = require('supertest'); const app = require('../server'); const
User = require('../models/User'); describe('POST /api/auth/signup', () => { it('TC-03-01: creates
student with hashed password', async () => { const res = await request(app).post('/api/auth/signup')
.send({ name: 'Alice', email: 'a@test.com', password: 'Secret123', role: 'student' });
expect(res.status).toBe(201); const user = await User.findOne({ email: 'a@test.com'
}).select('+password'); expect(user.password).not.toBe('Secret123'); // hashed
expect(res.headers['set-cookie']).toBeDefined(); }); it('TC-03-02: rejects duplicate email', async ()
=> { await User.create({ name: 'Bob', email: 'b@test.com', password: 'hashed', role: 'student' }); const
res = await request(app).post('/api/auth/signup').send({ name: 'Bob2', email: 'b@test.com',
password: 'pw' }); expect(res.status).toBe(400); expect(res.body.message).toBe('Email already in use');
}); });
```

17.2 Pytest Test: Diagnostic Analysis Service

```
# tests/test_diagnostic.py import pytest from services.diagnosticAnalysisService import (
computeModalityScores, detectStrongestWeakest ) def make_assessment(text_acc=80, audio_acc=60,
video_acc=90, inter_acc=70): return { 'textMode':
{'completed': True, 'accuracyPercent': text_acc, 'avgResponseTimeMs': 2000, 'readingOrWatchTimeSeconds': 120},
'audioMode':
{'completed': True, 'accuracyPercent': audio_acc, 'replayCount': 2, 'listeningDurationSeconds': 90},
'videoMode':
{'completed': True, 'accuracyPercent': video_acc, 'pauseCount': 3, 'readingOrWatchTimeSeconds': 200},
'interactiveMode': {'completed': True, 'accuracyPercent': inter_acc, 'interactionCount': 5}, } def
test_computeModalityScores_incomplete_mode_returns_zero(): assessment = make_assessment()
assessment['audioMode']['completed'] = False scores = computeModalityScores(assessment) assert
scores['audio'] == 0 def test_detectStrongestWeakest_picks_highest_and_lowest(): scores =
{'text': 80, 'audio': 60, 'video': 90, 'interactive': 70} result = detectStrongestWeakest(scores) assert
result['strongest'] == 'video' assert result['weakest'] == 'audio'
```

17.3 Jest Test: Adaptive Teacher clamp utility

```
// tests/adaptiveTeacher.test.js const { clamp } = require('../controllers/adaptiveTeacherController');
describe('clamp()', () => { it('TC-05-11: clamps above max to 100', () =>
expect(clamp(150)).toBe(100)); it('clamps below min to 0', () => expect(clamp(-5)).toBe(0));
it('returns value within range', () => expect(clamp(50)).toBe(50)); it('handles non-numeric → 0', () =>
expect(clamp(NaN)).toBe(0)); it('supports custom min/max', () => expect(clamp(3, 1, 5)).toBe(3)); });
```

17.4 Jest Test: Concept Map Service cache eviction

```
// tests/conceptMap.test.js const { generateConceptMapForContent } =
require('../services/conceptMapService'); const { chatCompletion } =
require('../services/grokService'); jest.mock('../services/grokService'); it('TC-11-10: uses fallback
when LLM throws', async () => { chatCompletion.mockRejectedValue(new Error('LLM down')); const content
= { _id: 'abc123', updatedAt: new Date(), title: 'Operating Systems', summary: 'Core OS concepts.',
blocks: [{ type: 'text', text: 'A process is a running program.' }], tags: ['os', 'kernel'] }; const map =
await generateConceptMapForContent(content); expect(map.model.usedLLM).toBe(false);
expect(map.model.fallbackReason).toMatch(/LLM down/);
expect(map.nodes.length).toBeGreaterThanOrEqual(1); });
```

17.5 Jest Test: Study Plan streak logic

```
// tests/studyPlan.test.js (integration with mongodb-memory-server) it('TC-06-07: increments streak on
new calendar day', async () => { const yesterday = new Date(); yesterday.setDate(yesterday.getDate() -
1); plan.lastStudiedAt = yesterday; plan.currentStreak = 3; await plan.save(); const res = await
request(app) .post('/api/study-plan/complete-session') .set('Authorization', `Bearer ${studentToken}`)
.send({ planId: plan._id, weekNumber:1, dayLabel:'Monday', sessionId:
plan.weeklyPlan[0].days[0].sessions[0]._id }); expect(res.body.data.currentStreak).toBe(4); });
```

Document Summary

NeuroLearn AI TDD Document

This document specifies **204 test cases** covering 12 backend modules (Node.js + Python), integration flows, and E2E scenarios. Tests follow RED-GREEN-REFACTOR TDD cycles. CI runs on every pull request via GitHub Actions with in-memory MongoDB and mocked AI providers. Target coverage: **80% lines, 75% branches**.

End of NeuroLearn AI Test-Driven Development Document — Version 1.0 — May 2025